Dodona 01 — cluster sanity

go / no-go, no load 2026-10-10 16:25 UTC

Objective

Does the cluster serve a real completion, and does it go THROUGH the control plane rather than past it?

Falsified by: orch_requests_total not moving, or a pool not answering

The promises

SLO	target	direction
TTFT p95	1.5s	max
turn p95	30s	max
preemptions	0	max
shed rate at target load	0.05	max
concurrent sessions	derived in 02	min

From benchmarking/slos.py. Session target is derived in 02.

Concurrency target not yet measured — run 02-capacity.

1. Endpoints

PASS gateway /metrics

PASS gateway /v1/models

PASS prefill /metrics

PASS decode /metrics

2. Model and pools

PASS gateway advertises: Qwen/Qwen2.5-7B-Instruct-AWQ

pool	running	waiting	prompt tok	gen tok
prefill	0	0	0	0
decode	0	0	0	0

prompt tok 0 on a pool means nothing was ever routed to it.

PASS prefill generated 0 tokens (expected near 0, max_tokens=1)

3. Capacity

quantity	value	where it comes from
KV per decode pod	4.30 GiB	pod log: Available KV cache memory
full-length seqs / pod	4.9	pod log: Maximum concurrency
decode replicas	1	kubectl get deploy vllm-decode
decode pool concurrency	4.9	seqs/pod x replicas

A pool this small is the point: a policy needs something to be scarce.

PASS pool of 4.9 is small enough for KV to bind

4. Does a request traverse the gateway

PASS orch_requests_total moved 66 -> 67

FAIL completion returned in 0.111s

ERROR HTTP 429: {"error": {"type": "rate_limit_error", "message": "tenant tokens/min cap"}, "via": "local"}

5. The KV hop

pool	prompt tok	prefix hits	queries	hit rate
prefill	0	0	0	n/a
decode	0	0	0	n/a

Decode loading blocks rather than recomputing shows as a high hit rate.

PASS decode processes fewer prompt tokens than prefill

6. Was anything scarce

pool	kv_cache_usage_perc	vs 0.90 threshold
prefill	0.0000	BELOW
decode	0.0000	BELOW

Idle cluster: near zero is expected. make load is what moves this.

An idle reading is not a verdict on the policy — see make load.

7. Where the work lands

7b. The slice budget

component	GiB
slice x util	10.80
weights	5.20
activations + CUDA graphs	1.30
KV cache	4.30

Overhead is budget minus weights minus KV. Measured, not predicted.

8. Latency, short prompt

FAIL no successful samples — see section 4

9. Latency vs prompt length

prompt tokens	p50 seconds

Warmup discarded, so these are cache-warm numbers.

FAIL all 5 prompt lengths completed

256 tokens DROPPED — HTTP 429: {"error": {"type": "rate_limit_error", "message": "tenant tokens/min cap"}, "via": "local

1,024 tokens DROPPED — HTTP 429: {"error": {"type": "rate_limit_error", "message": "tenant tokens/min cap"}, "via": "loc

4,096 tokens DROPPED — HTTP 400: {"error": {"type": "prompt_too_long", "message": "prompt > 8000 chars"}}

8,192 tokens DROPPED — HTTP 400: {"error": {"type": "prompt_too_long", "message": "prompt > 8000 chars"}}

12,288 tokens DROPPED — HTTP 400: {"error": {"type": "prompt_too_long", "message": "prompt > 8000 chars"}}

10. Grafana

- vLLM — KV, running/waiting, TTFT, prefix cache
  `http://127.0.0.1:31495/d/dodona-vllm?from=now-30m&to=now`
- Gateway — admit, shed, place, latency by stage
  `http://127.0.0.1:31495/d/dodona-gateway?from=now-30m&to=now`
- Router — pick, sticky, KV hop
  `http://127.0.0.1:31495/d/dodona-router?from=now-30m&to=now`
- KEDA — scaler value vs replicas

http://127.0.0.1:31495/d/dodona-keda?from=now-30m&to=now

HAMi — device memory vs the slice cap

http://127.0.0.1:31495/d/dodona-hami?from=now-30m&to=now

explore: KV usage per pod

http://127.0.0.1:31495/explore?left=%7B%22datasource%22%3A%20%22Prometheus%22%2C%20%22queries%22%3A%20%5B%7B%22

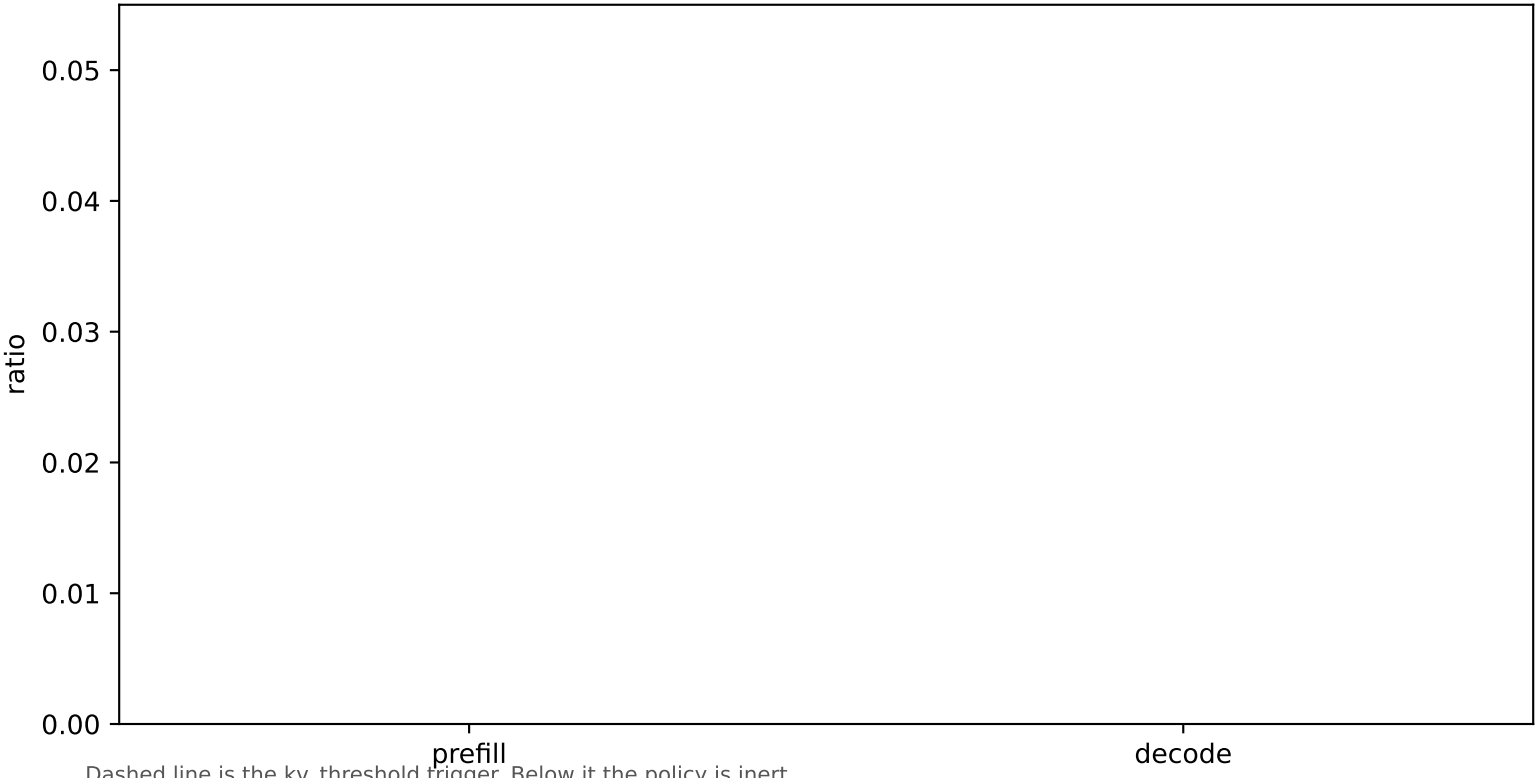
explore: queue depth per pod

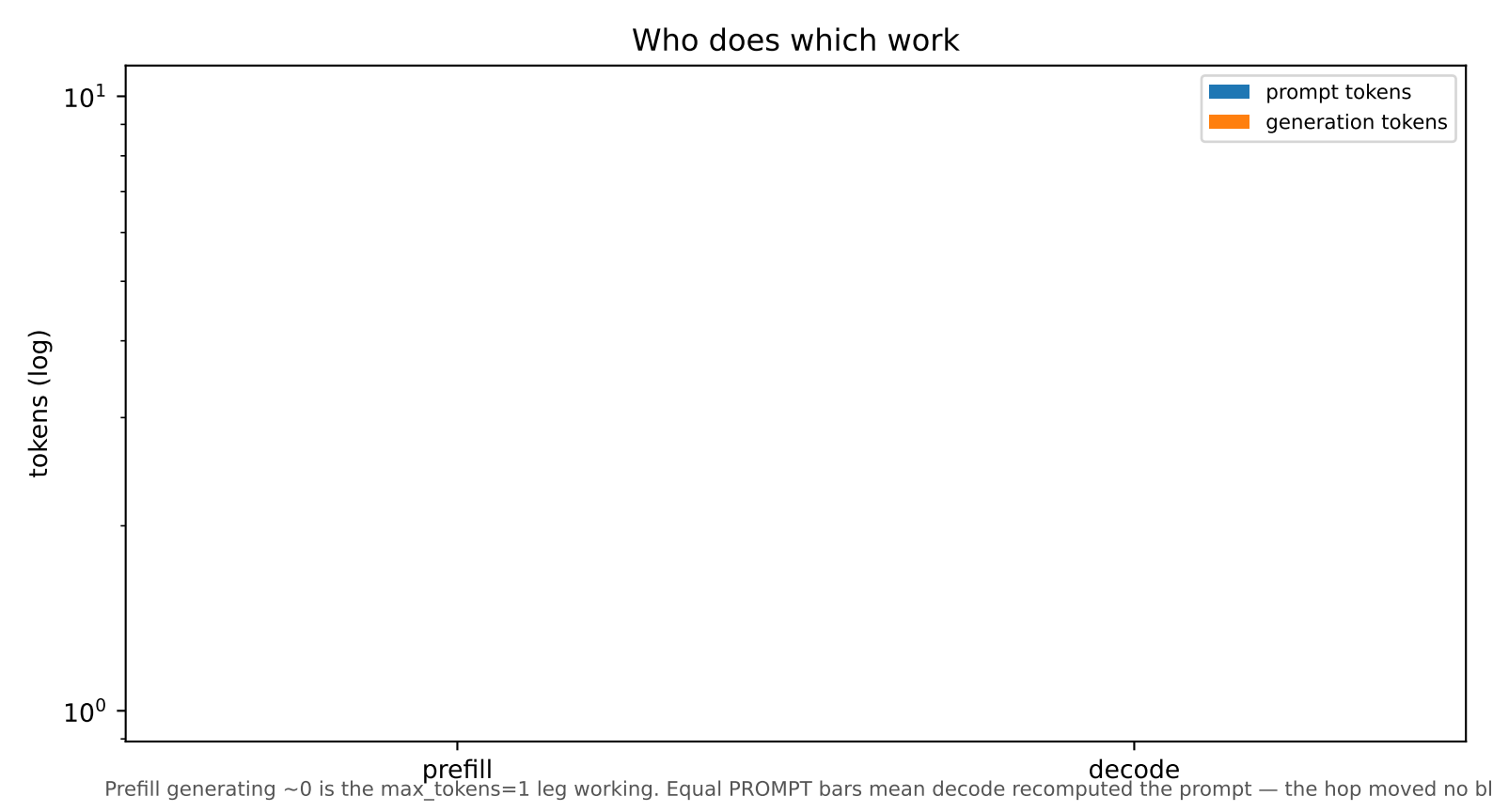
http://127.0.0.1:31495/explore?left=%7B%22datasource%22%3A%20%22Prometheus%22%2C%20%22queries%22%3A%20%5B%7B%22

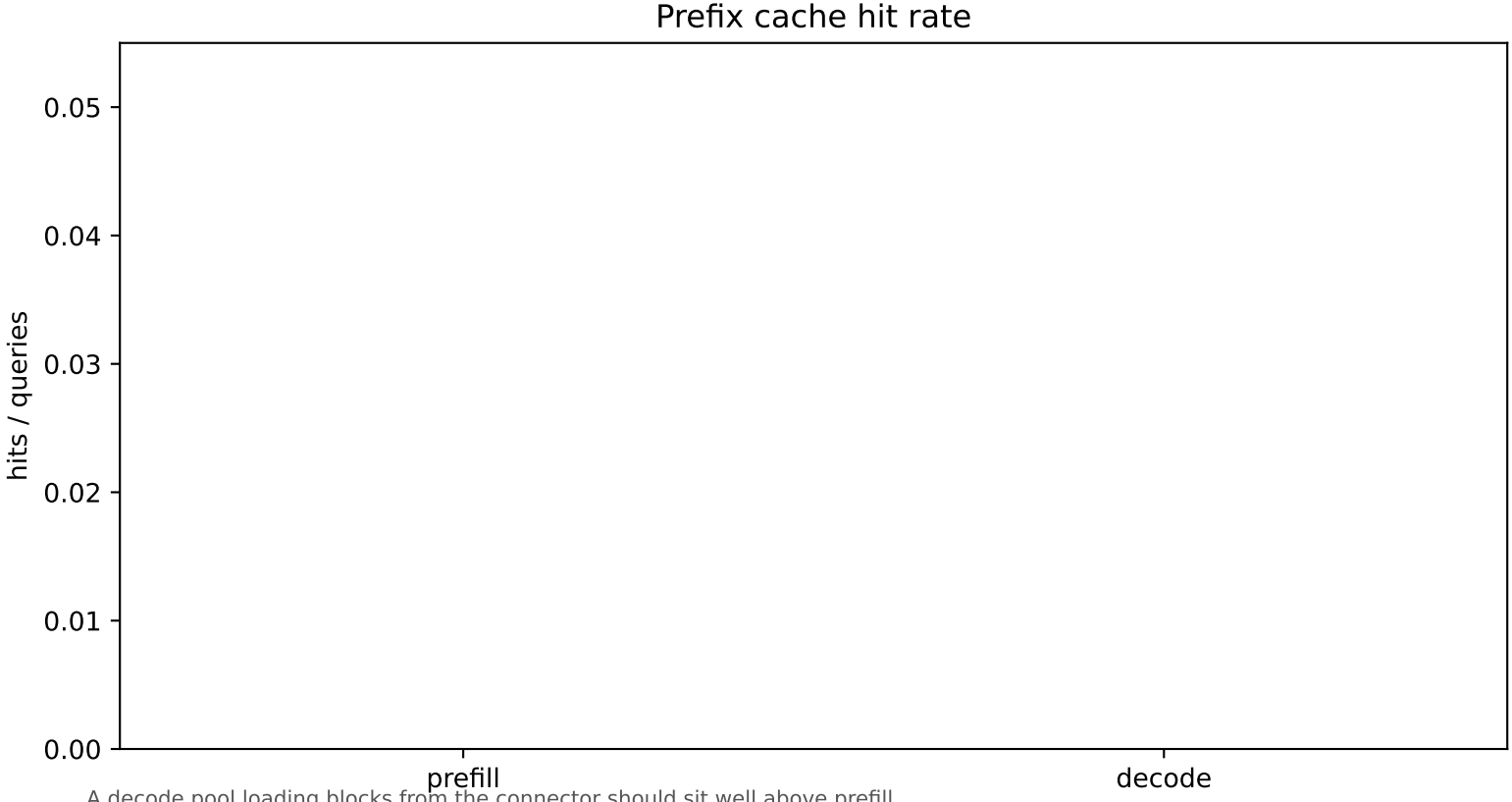
explore: gateway shed by reason

http://127.0.0.1:31495/explore?left=%7B%22datasource%22%3A%20%22Prometheus%22%2C%20%22queries%22%3A%20%5B%7B%22

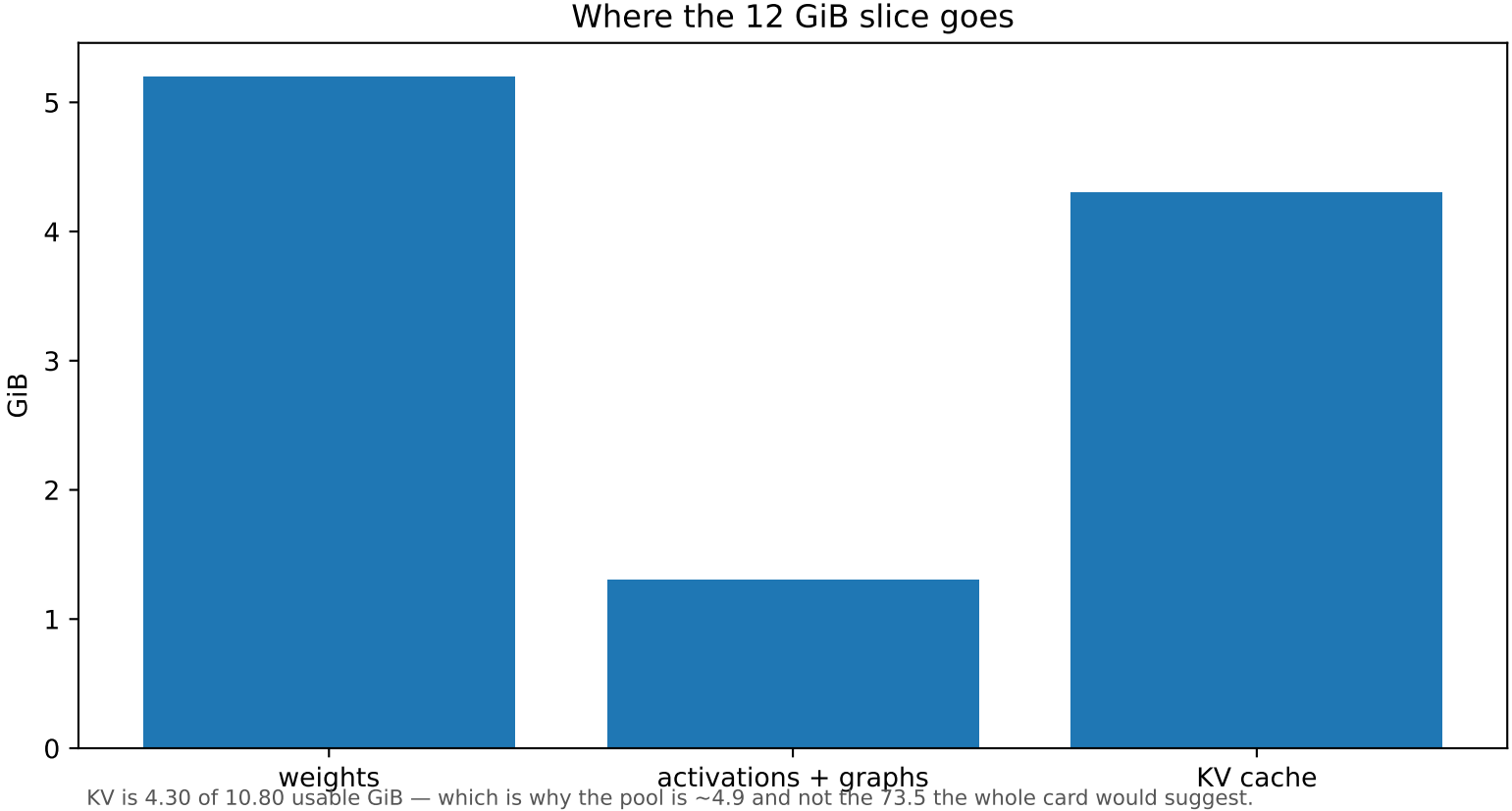
Grafana password is printed by make observability.


KV usage vs the 0.90 admission threshold
ratio
0.05
0.04
0.03
0.02
0.01
0.00
prefill
decode
Dashed line is the kv_threshold trigger. Below it the policy is inert.


Who does which work
prompt tokens
generation tokens
tokens (log)
10^1
10^0
prefill
decode
Prefill generating ~0 is the max_tokens=1 leg working. Equal PROMPT bars mean decode recomputed the prompt — the hop moved no bl



A decode pool loading blocks from the connector should sit well above prefill.


Where the 12 GiB slice goes
GiB
0
1
2
3
4
5
weights
activations + graphs
KV cache
KV is 4.30 of 10.80 usable GiB — which is why the pool is ~4.9 and not the 73.5 the whole card would suggest.